Fundamentals Primer

The basics behind GeoSemTwin-X, explained from zero

Cameras · 3D geometry · SLAM · mapping · deep learning · detection & segmentation · data association · uncertainty · exploration · evaluation · ROS 2 · tools

How this primer works

Each chapter covers one topic and follows the same pattern: the **intuition** in everyday words, a **picture**, the **key formula** (only the essential one), **where it appears in GeoSemTwin-X** (green box), and a short **check yourself** section with answers at the end. Read in order: later chapters build on earlier ones. No prior background is assumed beyond school maths (coordinates, vectors, probability).

Companion documents

This primer explains the *general* ideas. The *GeoSemTwin-X Beginner's Guide* explains how the project uses them, what was built and what the results mean. The *RUN_GUIDE.md* explains how to run the code.

Contents

1. Digital twins
2. Digital images and pixels
3. The pinhole camera model
4. 3D coordinates, rotations and transforms
5. Depth sensing and RGB-D cameras
6. From pixels to 3D: point clouds
7. Inertial measurement units (IMU)
8. Features, matching and RANSAC
9. Estimating motion: PnP, ICP and direct alignment
10. Odometry and drift
11. SLAM: keyframes, loop closure, pose graphs
12. Monocular scale and visual-inertial SLAM
13. Measuring SLAM accuracy: ATE and RPE
14. Representing 3D maps: voxels, TSDF, occupancy
15. Uncertainty: Gaussians, variance, weighting
16. Deep learning in one chapter
17. Object detection
18. Open-vocabulary detection (Grounding DINO)
19. Image segmentation (SAM)
20. Lifting 2D detections to 3D
21. Data association and the Hungarian algorithm
22. Semantic fusion and multi-view confirmation
23. Change detection
24. Exploration: frontiers, next-best view, A*
25. Evaluation: precision, recall, F1, IoU, mIoU
26. Experiments: baselines, ablations, sensitivity
27. ROS 2: nodes, topics, TF, RViz, Nav2
28. The tool chain: Linux, WSL, Python, CUDA, Hugging Face
29. Answers to "check yourself"

How the topics connect

Sensing (ch. 2–7)
How a camera turns the world into numbers, how depth and motion sensors work, and how to turn pixels back into 3D.

Localisation (ch. 8–13)
How a robot figures out where it is from images: matching, motion estimation, drift, SLAM, and how to measure accuracy.

Mapping (ch. 14–15)
How many noisy 3D measurements become one clean map, and how uncertainty decides what to trust.

Perception (ch. 16–19)
How neural networks find and outline objects in an image, even ones named only by text.

Semantic twin (ch. 20–24)
How 2D detections become persistent 3D assets, how changes are detected, and where to look next.

Evidence and systems (ch. 25–28)
How to evaluate fairly, and the software that runs it all on a robot.

1. Digital twins

A digital twin is a computer model of a real thing that is kept up to date with the real thing.

Intuition

Think of a shop's stock register. The register is useful only if it matches what is on the shelves. A digital twin is the same idea for a physical place or machine: a model that mirrors the real object's state, is updated from sensor data, and can be queried ("where is pallet 17?") or used to plan ("which aisle should the robot scan next?").

Levels of twins

Level	What it contains	Example
Digital model	Static geometry, never updated	CAD drawing of a warehouse
Digital shadow	Updated automatically from the real world (one direction)	Map updated each time a robot scans
Digital twin	Two-way: the model also informs actions in the real world	Twin tells the robot where to go next; manager acts on detected changes

Ingredients of a useful warehouse twin

- **Geometry:** a 3D map (walls, racks, floor) so locations mean something.
- **Semantics:** what things are (forklift, pallet), not just shapes.
- **Identity:** each physical object has one stable ID over time.
- **Versions / history:** snapshots over time so you can ask "what changed?".
- **Confidence:** how sure the system is about each fact.

In GeoSemTwin-X

The twin is a versioned registry of assets (ID, class, confidence, 3D box, first/last seen) plus a 3D map and an occupancy grid, stored as JSON per session and shown in an HTML dashboard. The next-best-view goal closes the loop back to the robot.

Check yourself

1.1 Why is a 3D model that is never updated not a digital twin? 1.2 Name two things a twin needs beyond a 3D map.

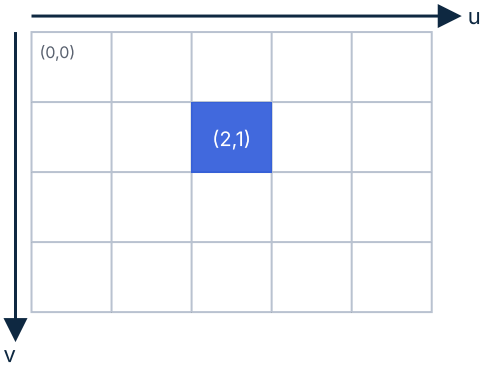
2. Digital images and pixels

An image is a grid of numbers. Everything in computer vision starts from this.

A colour image of width W and height H is an array of W × H **pixels**. Each pixel stores three numbers: red, green and blue (**RGB**), usually 0–255 each (8 bits). A greyscale image stores one brightness number per pixel.

Pixel positions are written as **(u, v)**: u counts columns from the left, v counts rows from the **top**. So v grows downwards, unlike the y-axis in school maths.

Resolution is W × H, e.g. 1920 × 1080 ("Full HD"). **Frame rate** is images per second (fps), e.g. 20 fps.



Pixel grid: origin at the top-left, u to the right, v downwards.

Depth images

A **depth image** has the same grid, but each pixel stores a distance (e.g. in millimetres, as a 16-bit number) instead of a colour. Zero usually means "no measurement". An **RGB-D** frame is a colour image plus an aligned depth image, so every coloured pixel also has a distance.

Video and compression

Video files (MP4, MKV) store frames compressed with codecs such as H.264/H.265 (lossy, for colour) or FFV1 (lossless, good for depth where exact values matter). Tools like **ffmpeg** decode them back into raw pixel arrays.

In GeoSemTwin-X

Staer frames are 1920×1080 at 20 fps; colour is H.265 MP4, depth and instance masks are 16-bit lossless MKV. We decode them with ffmpeg at half resolution (960×540) for speed.

Check yourself

2.1 How many numbers does a 640×480 RGB image contain? 2.2 Why is depth stored losslessly but colour can be compressed lossily?

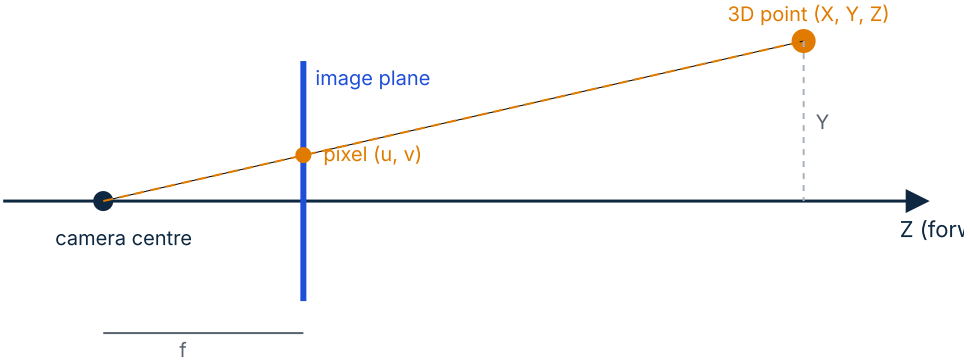
3. The pinhole camera model

How a 3D point in front of the camera ends up at a particular pixel.

Intuition

Imagine a dark box with a tiny hole. Light from a point in the world passes through the hole and hits the back wall. Points further to the side land further from the centre; points further away land closer to the

centre. That shrinking with distance is **perspective**. Real cameras use lenses, but the pinhole model describes them very well.



Similar triangles: the image offset is f·Y/Z. A point twice as far away appears half as far from the centre.

The projection formula

Using camera coordinates (X right, Y down, Z forward), a 3D point projects to pixel:

$$u = f_x \cdot X / Z + c_x \qquad v = f_y \cdot Y / Z + c_y$$

- **f_x, f_y**: focal length measured in pixels. Larger f = more zoom, narrower field of view.
- **c_x, c_y**: the principal point, where the optical axis hits the image (usually near the centre).

These four numbers are the **intrinsics**, usually packed into a 3×3 matrix **K**. They come from **calibration** (photographing a checkerboard) or from the sensor's data sheet.

Field of view

Horizontal field of view = 2 · atan(W / (2 f_x)). Example: W = 1920, f_x = 672 gives about 110°, a wide-angle camera.

Lens distortion

Real lenses bend straight lines (barrel or pincushion distortion). Calibration also estimates distortion coefficients so images can be "undistorted" before using the pinhole formula. Synthetic (rendered) cameras have zero distortion.

Scaling an image

If you resize an image by a factor s, multiply f_x, f_y, c_x, c_y by s too.

In GeoSemTwin-X

Staer intrinsics are f_x = f_y = 672.2, c_x = 960, c_y = 540 at full resolution (a ~110° camera). At half resolution all four are halved. Projection is used to draw persistent 3D asset boxes back into the camera image, and to check whether an asset should have been visible.

Check yourself

3.1 A point at (X, Y, Z) = (1, 0, 2) m with f_x = 500, c_x = 320: what is u? 3.2 What happens to u if the point moves to Z = 4 m?

4. 3D coordinates, rotations and transforms

How to describe where something is and which way it faces, and how to convert between viewpoints.

Coordinate frames

A **frame** is an origin plus three perpendicular axes. We use several: the **world** (or map) frame fixed to the building, the **camera** frame attached to the camera, the **IMU** frame, and so on. The same physical point has different coordinates in each frame.

Pose = rotation + translation

The **pose** of the camera is where it is (translation **t**, 3 numbers) and how it is turned (rotation **R**). Together, 6 degrees of freedom (6-DoF): 3 for position, 3 for orientation.

Rotation matrices

A rotation is a 3×3 matrix R whose columns are the rotated axes. It is **orthonormal** ($R^T R = I$, so its inverse is just its transpose) and has determinant +1. Rotating a vector is a matrix multiply: p' = R p.

Other ways to store rotations

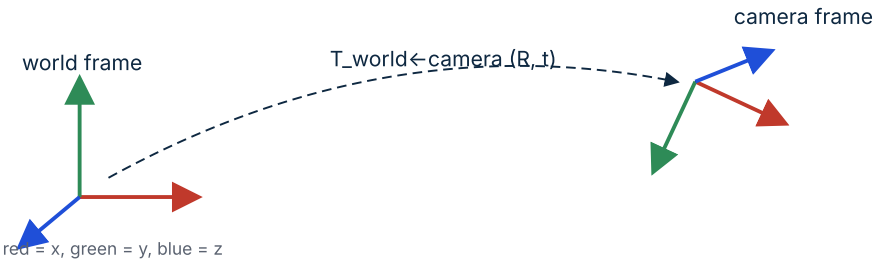
Form	Numbers	Notes
Rotation matrix	9	Easy to apply and combine; redundant
Euler angles (roll, pitch, yaw)	3	Intuitive, but order-dependent and can "gimbal lock"
Axis–angle / rotation vector	3	Direction = axis, length = angle; good for small rotations
Quaternion (w, x, y, z)	4	Compact, no gimbal lock, smooth interpolation ("slerp"); used in datasets and ROS

Homogeneous transforms

Rotation and translation are combined in one 4×4 matrix so that transforms chain by multiplication:

$$T = [\, R\ t\ ;\ 0\ 0\ 0\ 1 \,] \qquad p_{world} = T_{world \leftarrow camera} \cdot p_{camera} \qquad (p \text{ written as } [x, y, z, 1])$$

Chaining: $T_{A \leftarrow C} = T_{A \leftarrow B} \cdot T_{B \leftarrow C}$. Inverting: $T_{B \leftarrow A} = T_{A \leftarrow B}^{-1}$. Read the subscripts right to left, like a chain of conversions.



A pose is the transform that carries camera coordinates into world coordinates.

Axis conventions (a classic source of bugs)

Convention	x	y	z	Used by
Optical (computer vision)	right	down	forward	OpenCV, Open3D, ROS `*_optical` frames
OpenGL / graphics	right	up	backward	Rendering engines, Staer dataset
ROS body (REP-103)	forward	left	up	Robot bases, `base_link`

Converting OpenGL → optical flips y and z: multiply by diag(1, −1, −1). Using the wrong flip produces mirrored or upside-down maps.

In GeoSemTwin-X

Every frame's pose is a 4×4 camera-to-world matrix. Staer poses are OpenGL-style and are converted with diag(1, −1, −1); an early bug used diag(1, −1, 1) and mirrored the map. ROS publishes the TF `map → camera_optical`. VIGS-SLAM output uses quaternions, which we interpolate with slerp.

Check yourself

4.1 Why is the inverse of a rotation matrix its transpose? 4.2 If $T_{W \leftarrow C}$ is the camera pose, how do you convert a world point into camera coordinates?

---

5. Depth sensing and RGB-D cameras

How a camera can measure distance to every pixel, and why the measurement gets worse with

distance.

Three ways to measure depth

Principle	How it works	Examples
Stereo (passive or active)	Two cameras a known distance apart (baseline b). A point appears shifted between the two images by the **disparity** d. Depth Z = f · b / d. An IR projector adds texture to blank walls ("active stereo").	Intel RealSense D435i, ZED
Structured light	Project a known dot pattern; its distortion reveals depth (stereo between projector and camera).	Kinect v1 (used in TUM RGB-D)
Time of flight (ToF)	Measure how long emitted light takes to return.	Kinect v2/Azure, phone LiDAR

Why depth noise grows with distance

From Z = f·b/d, a fixed error of a fraction of a pixel in disparity causes a depth error that grows roughly with **Z^2**: at 1 m the error might be millimetres, at 8 m it can be tens of centimetres. So near measurements deserve more trust than far ones.

$$\sigma_Z \approx k \cdot Z^2 \quad \text{(k depends on the sensor's focal length, baseline and matching accuracy)}$$

Other practical facts

- **Range limits:** typical RGB-D cameras work from ~0.3 m to ~5–10 m.
- **Holes:** shiny, black, transparent surfaces and edges often give no depth (stored as 0).
- **Registration:** depth must be aligned to the colour camera ("aligned depth") so pixel (u, v) means the same thing in both.
- **Planar vs ray depth:** depth may be the distance along the optical axis (Z) or along the pixel's ray. Mixing them bends flat walls.
- **Inverse depth:** some datasets store 1/Z, which spends more precision on near objects.

LiDAR

A LiDAR spins laser beams and measures time of flight, giving accurate points out to tens of metres with almost constant error. It has much less detail per scan than a camera but is far more reliable for geometry and scale, which is why many robots combine cameras with LiDAR.

In GeoSemTwin-X

Staer depth is stored as inverse depth with a per-frame range, decoded to planar Z in metres. TUM uses a Kinect (structured light). The map gives each depth measurement a weight based on a Z^2 noise model, and a simulated multi-ring LiDAR checks for low obstacles in the exploration study.

Check yourself

5.1 If depth error at 2 m is 1 cm, roughly what is it at 6 m? 5.2 Why does a white wall cause trouble for passive stereo but not for active stereo?

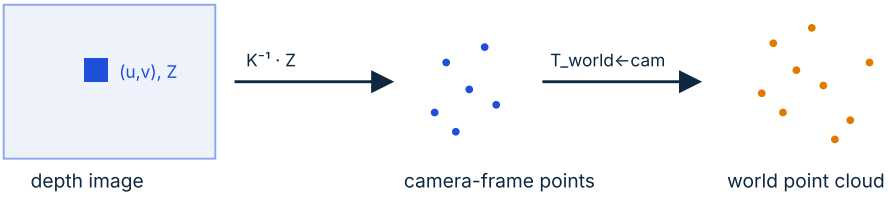
6. From pixels to 3D: point clouds

Reverse the pinhole formula and you can turn every depth pixel into a 3D point.

Back-projection

$$X = (u - c_x) \cdot Z / f_x \qquad Y = (v - c_y) \cdot Z / f_y \qquad Z = \text{depth}(u, v)$$

This gives the point in camera coordinates. Multiply by the camera pose $T_{\text{world}\leftarrow\text{camera}}$ to place it in the world. Doing this for all pixels of one frame gives a **point cloud**: a list of (x, y, z) points, often with a colour each.



Depth image → 3D points in the camera frame → world frame using the camera pose.

Working with point clouds

- **Downsampling:** keep one point per small cube (voxel grid), e.g. every 5 cm, to reduce millions of points to thousands.
- **Normals:** the surface direction at each point, estimated from neighbours; needed for point-to-plane ICP.
- **Outlier removal:** drop isolated points (sensor noise, "flying pixels" at object edges).
- **Connected components:** split a cloud into clusters of touching points; keep the cluster you want.
- **Robust extents:** describe a cluster's size with percentiles (e.g. 2nd–98th) instead of min/max so a few stray points do not inflate the box.

In GeoSemTwin-X

Every depth frame is back-projected and fused into a 5 cm voxel map. Detection masks are back-projected to give each object a 3D point cloud; we keep the largest nearby connected component and use 2–98% percentile extents for its 3D box.

Check yourself

6.1 Back-project pixel (u, v) = (420, 240) with Z = 2 m, $f_x = f_y = 500$, $c_x = 320$, $c_y = 240$. 6.2 Why use percentiles for an object's box?

7. Inertial measurement units (IMU)

A small chip that feels motion: how fast you rotate and how hard you accelerate.

What it measures

- **Gyroscope:** angular velocity (rad/s) around 3 axes. Integrate once to get orientation change.
- **Accelerometer:** specific force (m/s^2) on 3 axes, which includes gravity. At rest it reads about 9.81 m/s^2 upward; this tells you which way is down.

IMUs are fast (hundreds of readings per second) and work in the dark or with motion blur, but they drift quickly: to get position, acceleration must be integrated twice, so a small bias grows quadratically into large position errors.

Noise and bias

Each reading has random noise plus a slowly changing **bias** (offset). Data sheets list noise density and bias random walk; visual-inertial systems estimate the biases continuously.

Camera–IMU calibration

To combine camera and IMU, you need the rigid transform between them (**extrinsics**, often called T_{cb} or T_{BS}) and the time offset between their clocks.

Preintegration (idea)

Between two camera frames there may be 20 IMU readings. Visual-inertial systems summarise them into one relative motion estimate with its uncertainty ("IMU preintegration"), which becomes a constraint between the two frames.

Important limitation

An accelerometer only tells you about scale when there is actual acceleration. Moving at perfectly constant speed in a straight line produces no acceleration beyond gravity, so the IMU cannot tell "slow and small" from "fast and big". Chapter 12 explains why this matters.

In GeoSemTwin-X

The Staer dataset includes a 400 Hz IMU. We export it in the EuRoC CSV format (timestamp, gyro x/y/z, accel x/y/z) for VIGS-SLAM, with T_{cb} computed from the dataset's calibration files.

Check yourself

7.1 Why can't you get position from an IMU alone for long? 7.2 What does a stationary accelerometer measure?

8. Features, matching and RANSAC

To estimate motion, first find the same points in two images. Then ignore the wrong matches.

Keypoints and descriptors

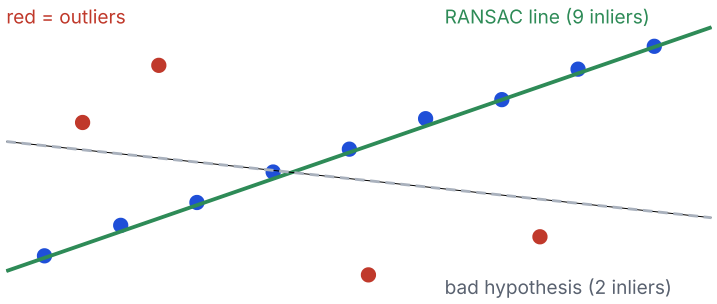
A **keypoint** is a distinctive spot, typically a corner, that can be found again from another viewpoint. A **descriptor** is a short fingerprint of the patch around it. Matching = finding the keypoint in image B whose descriptor is most similar to one in image A.

ORB (Oriented FAST and Rotated BRIEF) is a fast, free method: FAST finds corners, and BRIEF compares pairs of pixel brightnesses to build a 256-bit binary descriptor. Similarity is the **Hamming distance** (number of differing bits). ORB powers ORB-SLAM.

RANSAC: robust fitting with outliers

Many matches are wrong (repeated textures, moving objects). **RANSAC** (RANdom SAmple Consensus) handles this:

1. Pick the minimum random sample needed to fit a model (2 points for a line; 3–4 matches for a camera pose).
2. Fit the model; count how many other data points agree with it within a tolerance (**inliers**).
3. Repeat many times; keep the model with the most inliers; refit using all its inliers.



Least squares on all points would be pulled by the red outliers; RANSAC finds the line most points agree with.

The number of tries needed: $N = \log(1 - p) / \log(1 - w^s)$, where w is the inlier fraction, s the sample size and p the desired success probability. With 50% inliers, s = 4 and p = 0.99, about 72 tries.

In GeoSemTwin-X

When dense tracking fails, the SLAM falls back to ORB features matched to the keyframe, then PnP + RANSAC (chapter 9) to recover the pose ("PnP recovery").

Check yourself

8.1 Why is a corner a better keypoint than a point in the middle of a blank wall? 8.2 What is the minimal sample for fitting a line?

9. Estimating motion: PnP, ICP and direct alignment

Three families of methods to compute how the camera moved between two frames.

PnP (Perspective-n-Point)

Given n 3D points with known positions and where they appear in a new image, find the camera pose. Three points are the theoretical minimum (P3P, which can have several solutions); four or more with RANSAC gives a robust answer. With RGB-D data, the 3D positions come from the keyframe's depth.

ICP (Iterative Closest Point)

Align two point clouds directly:

1. For each point in cloud A, find the closest point in cloud B.
2. Compute the rigid transform that best aligns these pairs (closed-form, via SVD).
3. Apply it and repeat until it stops improving.

Point-to-plane ICP measures distance along the surface normal and converges faster. ICP needs a good starting guess and fails on geometry that does not constrain motion, e.g. sliding along a long flat corridor (the "degenerate" direction).

Direct (photometric) alignment

Instead of features, use all pixels: guess a motion, warp image A into B using depth, and measure the brightness difference. Adjust the motion to minimise that difference (Gauss–Newton optimisation over an image pyramid, coarse to fine). Works well with texture and small motions, poorly on blank surfaces or under lighting changes.

Hybrid RGB-D odometry

Combines a photometric term (colour consistency) and a geometric term (depth consistency) in one optimisation.

$$E(T) = \Sigma\,(I_B(\mathrm{warp}(p, T)) - I_A(p))^2 + \lambda \cdot \Sigma\,(\text{geometric depth error})^2$$

Information matrix

After optimisation, the curvature of E around the solution (the Hessian, J^TJ) tells how well each motion direction is constrained. This is the **information matrix**; its inverse is the covariance (uncertainty) of the estimate. A flat direction in E = high uncertainty.

In GeoSemTwin-X

Tracking uses Open3D's hybrid RGB-D odometry against the current keyframe, PnP + RANSAC as a fallback, and optionally ICP. The odometry's information matrix gives a per-step uncertainty σ that later down-weights uncertain frames in the map.

Check yourself

9.1 Why does ICP fail in a long featureless corridor? 9.2 What does a large value in the information matrix mean?

---

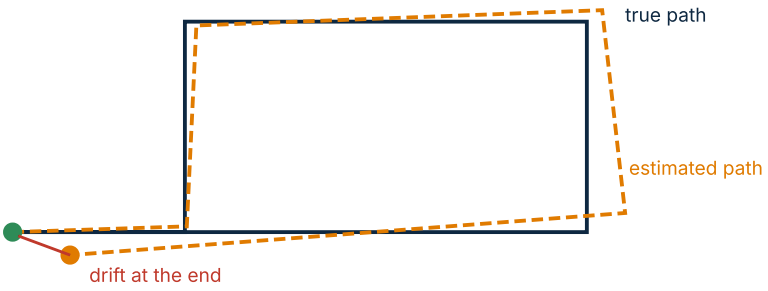
10. Odometry and drift

Chaining small motion estimates gives a trajectory, but errors pile up.

Intuition

Walk through a building with your eyes closed, counting steps and turns. After a few metres you are fine; after a few hundred, you could be in the wrong room. Each step's small error adds to the next. Visual **odometry** is the same: the pose at frame k is the product of all relative motions so far.

$$T_k = T_0 \cdot \Delta T_1 \cdot \Delta T_2 \cdot \ldots \cdot \Delta T_k$$



Small errors in each step add up: the estimated loop does not close where it started.

Why rotation errors are the worst

A heading error of just 1° means that after walking 20 m you are about 35 cm sideways. Rotation errors turn into growing position errors.

Fighting drift

- **Keyframes:** track against a fixed reference frame rather than the previous frame, so errors do not add every frame.
- **Motion prior:** start each estimate from a constant-velocity guess.
- **Sanity checks:** verify each estimate (e.g. how many depth pixels agree after warping) and fall back to another method if it looks wrong.
- **Loop closure:** recognise revisited places and correct the whole path (chapter 11).
- **Other sensors:** IMU, wheel odometry, LiDAR.

Drift percentage = end-point error ÷ distance travelled. 1% is good; above 10% indicates tracking trouble.

In GeoSemTwin-X

On TUM, odometry alone gives 7.3 cm error; adding loop closure brings it to 2.9 cm. On the straight Staer aisle, drift was 15% because the motion gave the trackers little to hold on to.

Check yourself

10.1 Why does tracking against a keyframe drift less than frame-to-frame tracking? 10.2 A robot travels 50 m and ends 1 m from the truth: what is the drift %?

---

11. SLAM: keyframes, loop closure, pose graphs

Simultaneous Localisation And Mapping: build a map and locate yourself in it at the same time.

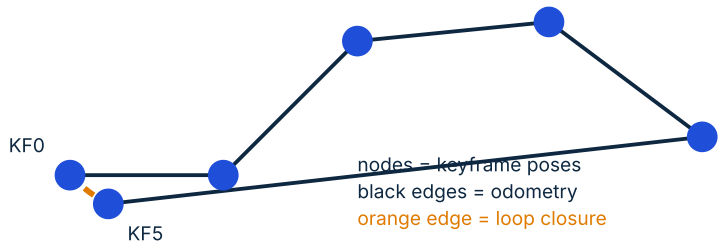
The chicken-and-egg problem

To know where you are, you need a map. To build a map, you need to know where you were. SLAM solves both together, using the map to correct the trajectory and the trajectory to build the map.

Typical SLAM pipeline

1. **Front end (tracking):** estimate motion frame by frame (chapter 9); decide keyframes.
2. **Loop detection:** when a new keyframe looks like an old one (similar appearance and geometry), propose a loop.
3. **Loop verification:** align the two frames; accept only if the alignment is consistent.
4. **Back end (optimisation):** correct all poses so they agree with both odometry and loops.

Pose graph optimisation



A pose graph: optimisation moves all nodes slightly so every edge is satisfied as well as possible, spreading the loop error along the path.

Each **node** is a keyframe pose; each **edge** is a measured relative motion with its information matrix. The optimiser minimises the total weighted disagreement:

$$\min \text{ over all poses } \quad \Sigma_{\text{edges}}\ e_{ij}{}^{T}\ \Omega_{ij}\ e_{ij} \quad (e = \text{mismatch between measured and current relative pose, } \Omega = \text{information})$$

Robust kernels reduce the influence of a wrong loop. After optimisation, all frames between keyframes are updated to follow their keyframe.

Bundle adjustment (BA)

A more detailed back end that jointly optimises camera poses and 3D feature points to minimise **reprojection error** (pixel distance between where a point is observed and where it projects). ORB-SLAM and VIGS-SLAM use BA; "after BA" trajectories are the refined ones.

Well-known systems

System	Type	Key idea
ORB-SLAM2/3	Feature-based	ORB features, BA, place recognition; mono, stereo, RGB-D (+IMU in v3)
DVO-SLAM	Direct RGB-D	Dense photometric + depth alignment, keyframe pose graph
ElasticFusion	Dense surfel map	Corrects the map itself (deformation) instead of a pose graph
VIGS-SLAM	Visual-inertial + Gaussian splatting	Camera + IMU tracking, map made of 3D Gaussians

In GeoSemTwin-X

Keyframes every 0.2 m or 12°, loop detection between keyframes, and Open3D pose-graph optimisation. An early bug used the wrong edge direction (Open3D's edges map source → target); fixing it plus adding a final keyframe roughly halved synthetic error.

Check yourself

11.1 What does a loop closure tell the optimiser? 11.2 What is the difference between a pose graph and bundle adjustment?

---

12. Monocular scale and visual-inertial SLAM

One camera cannot tell size; an IMU can, but only when the camera accelerates.

Scale ambiguity

A photo of a toy car 1 m away can look identical to a real car 10 m away. With a single camera (monocular), the whole scene and trajectory can be scaled up or down without changing any image. Monocular SLAM therefore recovers the map only "up to scale".

How scale is recovered

- **Depth sensors (RGB-D) or stereo:** measure metric distance directly.
- **IMU:** the accelerometer measures acceleration in m/s^2. Comparing it with the camera's (unscaled) acceleration reveals the scale factor, the gravity direction and the IMU biases. This is done during **IMU initialisation**.
- **Known object sizes** or wheel odometry.

Observability: when the IMU cannot help

Scale is visible to the IMU only through acceleration. In **constant-velocity, straight-line motion** the true acceleration is zero, so every scale fits the data equally well: scale becomes **unobservable**. Pure rotation without translation causes similar problems for depth. Visual-inertial systems need "excitation": changes of speed and direction, especially at the start.

Real example

The Staer walkthrough used in GeoSemTwin-X moves exactly 5.0 cm per frame with almost zero rotation. VIGS-SLAM's trajectory came out 2.3× wrong in scale, as the theory predicts.

3D Gaussian splatting (the "GS" in VIGS-SLAM)

A newer way to represent a scene: millions of small, coloured, semi-transparent 3D Gaussian "blobs". They can be rendered very fast by projecting ("splatting") them onto the image, and optimised so renders match the camera images. It gives photo-realistic maps, but the core localisation problems above still apply.

In GeoSemTwin-X

VIGS-SLAM (mono + IMU) is the external baseline. Our RGB-D SLAM has depth, so its scale error on the same sequence was smaller (1.3× vs 2.3×). The roadmap adds LiDAR, which measures metric distance directly regardless of motion.

Check yourself

12.1 Why can't a single camera measure scale? 12.2 Describe a motion that helps IMU initialisation and one that does not.

---

13. Measuring SLAM accuracy: ATE and RPE

Compare the estimated trajectory with ground truth from a motion-capture system.

Ground truth

Benchmarks record the true camera pose with an external system (e.g. motion-capture cameras tracking markers, or the simulator's exact pose in synthetic data). Timestamps are matched between estimate and ground truth.

Alignment first

The estimate starts in its own coordinate frame, so before comparing we find the rigid transform (rotation + translation, "SE(3)") that best overlays the two trajectories (Horn / Umeyama method). For monocular systems we also allow a scale ("Sim(3)"); the scale found tells you how wrong the system's scale was.

ATE: absolute trajectory error

$$\mathrm{ATE}_{\mathrm{RMSE}} = \sqrt{\left((1/N) \, \Sigma_i \, \| \, t_i^{\mathrm{est,\ aligned}} - t_i^{\mathrm{gt}} \, \|^2 \right)}$$

One number for global accuracy: the typical distance between where the system thinks the camera was and where it really was.

RPE: relative pose error

Compare the motion over a fixed interval (e.g. 1 s) between estimate and ground truth. Measures local drift and is reported as translation (m) and rotation (degrees). It does not need global alignment.

Metric	Answers	Sensitive to
ATE	How far off is the whole path?	Accumulated drift, loop closure quality
RPE	How accurate is each short motion?	Front-end tracking quality
Sim(3) scale	Is the map the right size?	Monocular / IMU scale problems

In GeoSemTwin-X

TUM fr1/desk: ATE 2.9 cm (ORB-SLAM2 1.6, ElasticFusion 2.0, DVO-SLAM 2.1 cm as published). Staer: ATE 2.71 m with Sim(3) scale 1.30 (ours) vs 4.98 m and scale 2.30 (VIGS-SLAM).

Check yourself

13.1 Why must trajectories be aligned before computing ATE? 13.2 What would a Sim(3) scale of 1.0 mean?

---

14. Representing 3D maps: voxels, TSDF, occupancy

Many overlapping point clouds must become one compact, clean map.

Voxel grids

Divide space into small cubes (**voxels**, e.g. 5 cm). Each voxel stores a summary: an average point and colour, a count, a label. Storing only occupied voxels in a hash table ("sparse" or "scalable" grid) keeps memory small.

TSDF fusion

A **Truncated Signed Distance Function** stores, for each voxel near a surface, the signed distance to the surface (positive in front, negative behind), truncated to a small band. Each new depth frame updates voxels with a weighted running average, which averages out noise. The surface is extracted where the value crosses zero (**marching cubes**). KinectFusion popularised this.

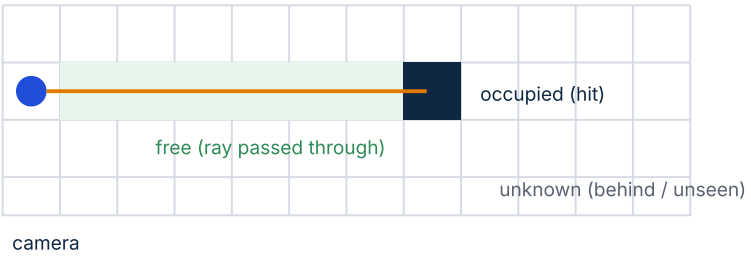
$$D_{\mathrm{new}} = (W \cdot D + w \cdot d) \, / \, (W + w) \qquad W_{\mathrm{new}} = W + w$$

Occupancy grids

For navigation we need to know which space is **free**, **occupied** or **unknown**. For each depth measurement, cast a ray from the camera to the measured point: voxels along the ray are evidence for free, the end voxel is evidence for occupied. Evidence is accumulated in **log-odds**:

$$l = \log(\, p \,/\, (1 - p)\,) \qquad l \leftarrow l + l_{occ}\ \text{(hit)} \quad \text{or} \quad l \leftarrow l + l_{free}\ \text{(pass-through)}, \text{ clamped to } [l_{min},\, l_{max}]$$

Log-odds turn Bayesian updates into additions; clamping lets the map change when the world changes. Unknown cells (never observed) stay at 0 (p = 0.5): this distinction between "free" and "not observed" is crucial.



Ray casting: cells along the ray become more "free", the end cell more "occupied"; unseen cells stay unknown.

Traversability

Project the occupancy grid down to a 2D floor map: a cell is drivable if the floor there was observed free and there is no obstacle within the robot's height. Planners search paths over this grid.

In GeoSemTwin-X

Open3D's TSDF volume returned empty maps in our setup, so we wrote `FusedMap`: a sparse 5 cm voxel map with an uncertainty-weighted running average (chapter 15). A separate log-odds occupancy map gives free/occupied/unknown, used for traversability, exploration and verifying changes.

Check yourself

14.1 Why is "unknown" different from "free" for change detection? 14.2 What does p = 0.5 correspond to in log-odds?

---

15. Uncertainty: Gaussians, variance, weighting

Every measurement is uncertain. Good systems know how uncertain, and trust measurements accordingly.

The Gaussian (normal) distribution

Most sensor noise is modelled as a bell curve with mean μ and standard deviation σ. About 68% of values fall within ±1σ, 95% within ±2σ, 99.7% within ±3σ. **Variance** is σ^2. In several dimensions, a **covariance matrix** Σ describes the spread in each direction and how directions are correlated.

Combining measurements: inverse-variance weighting

If you measure the same quantity several times with different reliabilities, the best (minimum-variance) estimate weights each by $1/\sigma^2$:

$$\hat{x} = \Sigma\, w_i\, x_i \,/\, \Sigma\, w_i, \quad w_i = 1 \,/\, \sigma_i^2 \qquad \sigma_{\hat{x}}^2 = 1 \,/\, \Sigma\, w_i$$

Example: a depth of 4.00 m (σ = 1 cm) and 4.10 m (σ = 10 cm): weights 1 and 0.01, so the fused value is about 4.001 m. The poor measurement barely matters, as it should.

Information = inverse covariance

The **information matrix** $\Omega = \Sigma^{-1}$. Large information = small uncertainty. Information from independent measurements simply adds, which is why optimisers and fusion work with it.

Gating and the Mahalanobis distance

To decide whether a new observation could belong to an existing estimate, measure the distance in units of uncertainty:

$$d_M^2 = (x - \mu)^T\, \Sigma^{-1}\, (x - \mu)$$

A "4σ gate" accepts a match only if the observation is within about 4 standard deviations. This adapts automatically: when the robot is uncertain, the gate widens.

Propagation of uncertainty

Uncertainty flows through calculations: a camera-pose error of 5 cm shifts every point seen from that pose by about 5 cm, plus the depth noise. Chained motions add their covariances (roughly), which is why drift grows.

In GeoSemTwin-X

Map weight $w = 1/\sigma_z(z)^2 \cdot \exp(-\sigma_{pose}/0.1)$: inverse-variance weighting for depth, times a penalty for uncertain poses; frames with $\sigma_{pose} > 0.15$ m are skipped. Asset matching across sessions uses a 4σ gate.

Check yourself

15.1 Fuse 2.0 m (σ = 2 cm) and 2.1 m (σ = 2 cm). 15.2 Why does the gate widen when pose uncertainty grows?

16. Deep learning in one chapter

Just enough to understand what Grounding DINO and SAM are.

Neural networks

A neural network is a function with millions of adjustable numbers (**weights**). Layers multiply inputs by weights, add biases and apply simple non-linear functions. **Training** shows the network many examples and nudges the weights (by gradient descent and back-propagation) to reduce a loss, e.g. the error between predicted and true boxes. **Inference** means just running the trained network on new data; that is all GeoSemTwin-X does: no training.

Convolutional networks (CNNs)

Slide small filters over the image to detect edges, then textures, then parts, then objects in deeper layers. Good at local patterns; the classic choice for vision for a decade.

Transformers and attention

Split the image into patches (or a sentence into tokens), turn each into a vector, and let every element look at every other one through **attention**:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^{T} / \sqrt{d}) \cdot V$$

Each element asks a question (query Q), compares it with every element's key K, and gathers a weighted mix of their values V. This captures long-range context ("this blue shape is next to forks, so it is probably a forklift"). Vision Transformers (ViT) and Swin Transformers apply this to images. **Cross-attention** between image and text lets a model link words to image regions.

Pretraining and foundation models

Large models are pretrained on huge datasets (billions of images or image–text pairs) and then used for many tasks without retraining. These are often called **foundation models**. Grounding DINO and SAM are examples.

GPUs and numerical precision

- Networks are mostly matrix multiplications, which GPUs do in parallel thousands at a time. **CUDA** is NVIDIA's platform for running them.
- **VRAM** (GPU memory) must hold the weights and intermediate results.
- **fp32** uses 32 bits per number; **bf16** (bfloat16) uses 16 bits with the same range as fp32 but less precision, roughly halving memory and speeding up inference with negligible accuracy loss.

In GeoSemTwin-X

Both perception models run pretrained, in inference only, on the RTX 3500 Ada (12 GB); SAM runs in bf16. Peak GPU memory was 2.9 GB and detection plus segmentation took 354 ms per keyframe.

Check yourself

16.1 What is the difference between training and inference? 16.2 Why does bf16 reduce memory use?

---

17. Object detection

Find each object in an image: a box, a class and a confidence.

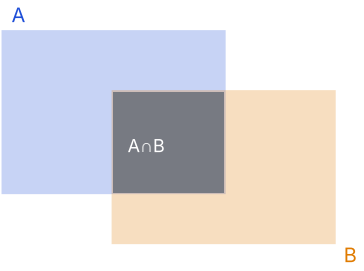
Output of a detector

For each object: a **bounding box** (x_{min}, y_{min}, x_{max}, y_{max}), a **class label** (e.g. "pallet"), and a **confidence score** between 0 and 1. A **threshold** discards low-confidence detections: lower thresholds find more objects but also more false ones.

IoU: how much two boxes overlap

$$\text{IoU} = \text{area}(A \cap B) \,/\, \text{area}(A \cup B)$$

IoU is 1 for identical boxes and 0 for non-overlapping ones. A detection usually counts as correct if its IoU with a true box is at least 0.5. The same idea applies to masks (pixels) and to 3D voxels.



Intersection (dark) over union (everything coloured).

Non-maximum suppression (NMS)

Detectors often fire several overlapping boxes on one object. NMS keeps the highest-scoring box and removes others that overlap it by more than an IoU threshold. Newer transformer detectors (DETR, DINO) predict a set of unique boxes directly and need little or no NMS.

Closed-set vs open-vocabulary

Classic detectors (e.g. YOLO trained on COCO) know a fixed list of classes from training; adding "pallet truck" means collecting and labelling images and retraining. Open-vocabulary detectors accept class names as text (chapter 18).

Typical failure modes

- Small or distant objects (few pixels).
- Partial occlusion; objects cut off at the image border.
- Confusing similar classes (pallet vs pallet truck).
- Inconsistent labels for the same object across frames: this is why multi-frame fusion is needed.

In GeoSemTwin-X

Detection runs on keyframes only. The threshold study varied the detector confidence from 0.20 to 0.45; 0.20 gave the best F1 for the twin.

Check yourself

17.1 Two 10×10 boxes overlap in a 5×10 region: what is the IoU? 17.2 Why does a per-frame detector alone produce duplicates in a map?

---

18. Open-vocabulary detection (Grounding DINO)

Detect objects named by any text, without retraining.

Idea

Train on enormous numbers of images paired with captions and phrases, so the model learns to "ground" words in image regions. At use time you type the classes you want: `"forklift. pallet. cardboard box. pallet truck."`, and it returns boxes with the matching phrase and a score.

How Grounding DINO works (high level)

1. An **image backbone** (Swin Transformer) turns the image into feature maps.
2. A **text encoder** (BERT) turns the prompt into one vector per word or phrase.
3. A **feature enhancer** lets image and text features attend to each other (cross-attention).
4. **Language-guided queries** pick the image regions most related to the text.
5. A **decoder** (DINO-style transformer) outputs boxes, each scored against every phrase.

Two thresholds control the output: a **box threshold** (is there an object?) and a **text threshold** (does it match this phrase well enough?).

Strengths and limits

Strengths	Limits
New classes by typing words; no labelled data needed	Lower accuracy than a detector trained for the specific site
Understands descriptions ("red forklift")	Struggles with small or distant objects, unusual viewpoints
Pairs naturally with SAM for masks	Slower than compact closed-set detectors; "tiny" model trades accuracy for speed

In GeoSemTwin-X

We use `IDEA-Research/grounding-dino-tiny` through Hugging Face Transformers, with warehouse prompts. Low recall on the Staer warehouse (19% at best) is mainly this detector missing small, distant boxes, which is why a larger or fine-tuned detector is on the roadmap.

Check yourself

18.1 What does "grounding" mean here? 18.2 What would you do to add a new class "roll cage" with a closed-set detector vs Grounding DINO?

---

19. Image segmentation (SAM)

Go from a box to the exact pixels of the object.

Kinds of segmentation

Type	Output	Example question
Semantic	A class for every pixel; objects of the same class are not separated	Which pixels are "pallet"?
Instance	A separate mask per object	Which pixels belong to pallet #3?
Panoptic	Both: instances for objects, classes for background ("stuff")	Full scene parsing

Segment Anything (SAM, SAM 2, SAM 2.1)

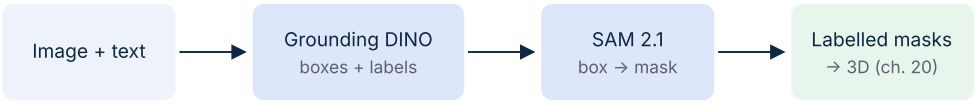
SAM is a promptable segmentation model from Meta: give it an image and a **prompt** (a point, a box or a rough mask) and it returns a high-quality mask of the object, for almost any kind of object. It has three parts:

- **Image encoder** (a Vision Transformer): computes image features once per image (the expensive part).
- **Prompt encoder**: encodes points or boxes.
- **Mask decoder**: a light network that combines both and outputs masks with quality scores.

SAM was trained on SA-1B (about 1.1 billion masks). **SAM 2** extends it to video with a memory of previous frames so it can track a mask over time; SAM 2.1 is an improved release. Models come in sizes (tiny, small, base+, large).

"Grounded SAM": detection + segmentation

SAM does not name objects. Combining an open-vocabulary detector (boxes + names) with SAM (precise masks) gives labelled instance masks for any text-described class.



The perception chain used in GeoSemTwin-X.

In GeoSemTwin-X

SAM 2.1 (tiny) segments each Grounding DINO box. Masks, not boxes, are lifted to 3D, which keeps background pixels out of the object's point cloud.

Check yourself

19.1 Why lift masks rather than whole boxes to 3D? 19.2 What does SAM need as input besides the image?

---

20. Lifting 2D detections to 3D

Combine a mask, the depth image and the camera pose to place an object in the world.

Steps

1. Take the pixels inside the mask.
2. Back-project each with its depth (chapter 6) and transform with the camera pose → a 3D point cloud of the object.
3. Clean it: drop pixels without depth or beyond a range limit; keep the largest connected cluster near the camera (mask edges often "leak" onto the background far behind).
4. Apply common-sense checks, e.g. a forklift must touch the floor.
5. Summarise: centroid, robust extent (3D box), voxel set, size, label, confidence, and the viewpoint it was seen from.

Why this is error-prone

- **Mask leakage:** a few background pixels can drag the box metres backwards.
- **Partial views:** you only see the front of an object, so its 3D box is too thin until other views add more points.
- **Pose error:** the whole object shifts with the camera pose error.
- **Depth noise:** far objects are noisier (Z^2 rule).

Each of these makes a single observation unreliable, which motivates fusing many observations (chapters 21–22).

In GeoSemTwin-X

`make_segments` in `gasf.py` does exactly this, with a floor-contact rule for forklifts and pallet trucks and a minimum score.

Check yourself

20.1 Why keep the connected component nearest the camera? 20.2 Why is a single observation's 3D box often too small?

21. Data association and the Hungarian algorithm

Which new observation belongs to which known object?

The problem

At each keyframe you have M new observations and N known objects. You must decide which pairs are the same physical object, which observations are new objects, and which objects were not seen. This is **data association**, the heart of tracking and of building any registry.

Score table and assignment

Compute a score (or cost) for every observation–object pair, e.g. based on overlap, distance and label agreement. Then choose a one-to-one matching with the best total score. Greedy matching (best pair first) can make poor global choices; the **Hungarian algorithm** (Kuhn, 1955) finds the optimal assignment in polynomial time ($O(n^3)$).

	obj A	obj B	obj C
obs 1	0.9	0.8	0.1
obs 2	0.85	0.2	0.0
obs 3	0.0	0.1	0.7

Greedy: 1→A (0.9), 2→? (only 0.2 left), 3→
total 0.9 + 0.2 + 0.7 = 1.8

Hungarian: 1→B, 2→A, 3→C
total 0.8 + 0.85 + 0.7 = 2.35 (best)

Greedy takes the single best pair first and is forced into a bad match later; the Hungarian algorithm optimises the total.

Gating and new objects

- **Gate:** pairs below a minimum score (or beyond a Mahalanobis distance) are not allowed to match.
- Observations left unmatched start **new candidate** objects.
- Objects left unmatched get a **miss** if they should have been visible.

Classic tracking approaches

Approach	Matches by	Weakness
Centroid / nearest-neighbour tracking	Distance between centres	Drift moves centres apart → duplicates
IoU tracking	Box overlap	Fails when overlap is small (fast motion, drift)
Kalman-filter tracking (e.g. SORT)	Predicted position + gate	Built for moving objects in 2D video
Multi-cue (GASF)	Overlap + centre + shape + label	Needs weights; more computation (still only 3 ms)

In GeoSemTwin-X

GASF scores pairs with S = 0.25·sem + 0.35·overlap + 0.20·centroid + 0.20·geo and uses SciPy's `linear_sum_assignment` (Hungarian) per keyframe and between twin versions.

Check yourself

21.1 What happens to an observation that matches nothing? 21.2 Why does centroid tracking produce duplicates under drift?

---

22. Semantic fusion and multi-view confirmation

Turn many noisy observations into one reliable object with a stable label.

Label fusion

An object may be called "pallet truck" (score 0.6), then "pallet" (0.3), then "pallet truck" (0.7). Keep a running score per class, adding each observation's confidence; the label is the class with the highest total. This is a simplified Bayesian update: high-confidence observations count more.

Geometry fusion

Merge point sets from all observations; recompute centroid and a robust box. The object becomes more complete and less noisy as views accumulate.

Multi-view confirmation

A single detection might be a hallucination (a shadow called "box"). Requiring an object to be seen several times (**hits**) from different **viewpoints** before trusting it removes most false positives. The trade-off: objects seen only once are never confirmed, which lowers recall.

Visibility-aware misses and pruning

If an object should be visible (inside the image, in range, not hidden) but is not detected, count a miss; repeated misses remove unconfirmed candidates. Objects outside the view are left alone.

Merging fragments

Large objects are sometimes detected in pieces (front and back of a forklift) that become two candidates. If they touch and their combined size is plausible for one object, merge them.

Geometry first, semantics second

Labels flicker between frames; geometry is more stable. Letting geometric evidence decide "same object or not", and the label decide "what it is", is the core design principle of GASF.

In GeoSemTwin-X

Confirmation needs ≥2 hits from ≥2 views; labels are confidence-weighted. The ablations show each part matters: without geometry F1 drops from 0.81 to 0.34; without the multi-view gate false positives rise 7×; without merging duplicates rise from 0.11 to 0.45 per asset.

Check yourself

22.1 What does multi-view confirmation trade away? 22.2 Why weight labels by confidence?

23. Change detection

Compare two versions of the twin and report what really changed.

The naive approach and why it fails

Match assets between session 1 and session 2. Unmatched old assets = "removed", unmatched new = "added", matched but far apart = "moved". The problem: an asset is often unmatched simply because the camera **did not look there** this time, or a drifted pose makes it appear moved. Naive change detection produces many **false alarms**.

Verification with occupancy (evidence of absence)

Claim	Evidence required
Removed	The old location was observed **free** in the new session (rays passed through it)
Added	The new location was observed **free** in the old session
Moved	Occupancy confirms the object at the new place now and at the old place before
Otherwise	**Unverified**: not enough observation to decide

"Absence of evidence is not evidence of absence": only space that was actually seen empty can confirm a removal. This is exactly why the occupancy map distinguishes free from unknown (chapter 14).

Measuring it

Precision (how many reported changes are real) and recall (how many real changes are found), on a scene with known, scripted changes.

In GeoSemTwin-X

On the synthetic warehouse (6 scripted changes), verification raised precision from 0.56 to 1.00 at unchanged recall 0.83. On the static Staer revisit, 0 false alarms; unobserved assets are listed as "unverified".

Check yourself

23.1 Why is "not detected" not enough to call an asset removed? 23.2 What does verification typically cost: precision or recall?

24. Exploration: frontiers, next-best view, A*

Decide where the robot should go next to learn the most.

Frontier-based exploration

A **frontier** is the boundary between known free space and unknown space (Yamauchi, 1997). Driving to a frontier reveals new area. The classic strategy: go to the nearest frontier, repeat until none remain.

Simple and effective.



Frontiers (orange) separate explored free space from the unknown.

Next-best view (NBV)

Instead of only frontiers, sample candidate positions and orientations and estimate the **information gain** of each: how much unknown space, uncertain surface, or unresolved objects it would reveal, minus a cost for travel distance. Pick the best, go there, repeat. NBV is more expensive but can target what matters (e.g. confirming ambiguous assets).

$$\mathrm{U(view)} = \mathrm{gain(view)} \cdot \mathrm{e}^{-\lambda \cdot \mathrm{distance}}$$

Path planning with A*

To reach a goal on the traversability grid, A* explores cells in order of f = g + h, where g is the cost so far and h a heuristic estimate to the goal (e.g. straight-line distance). If h never overestimates, A* finds the shortest path, while exploring far fewer cells than a blind search.

Replanning and safety

Maps change as the robot moves. When a new obstacle blocks the path (seen e.g. by a LiDAR scan at low height), the robot replans. Collisions and the number of replans are standard safety metrics.

In GeoSemTwin-X

$U = (w_u \cdot \text{unknown} + w_s \cdot \text{uncertain} + w_a \cdot \text{ambiguous}) \cdot e^{-0.15 \cdot d}$, A* on traversability, multi-ring LiDAR replanning. Compared with nearest frontier and random goals on an 80 m budget: NBV 27.6% coverage, frontier 28.0%, random 19.0%. The goal is published for Nav2.

Check yourself

24.1 What is a frontier? 24.2 Why must A*'s heuristic not overestimate?

25. Evaluation: precision, recall, F1, IoU, mIoU

How to turn "it looks good" into numbers others can trust.

The confusion counts

	Really there	Not really there
System says yes	True positive (TP)	False positive (FP)
System says no	False negative (FN)	True negative (rarely counted in detection)

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \qquad \text{Recall} = \text{TP} / (\text{TP} + \text{FN}) \qquad \text{F1} = 2 \cdot \text{P} \cdot \text{R} / (\text{P} + \text{R})$$

- **Precision:** "when it says something, is it right?" High precision = few false alarms.
- **Recall:** "does it find everything?" High recall = few misses.
- **F1:** balances both; high only when both are high.

They trade off: lowering a detection threshold raises recall and usually lowers precision. Always report both.

Example

22 real assets; the system reports 22 objects, of which 18 are correct and 2 are duplicates and 2 false. Precision = 18/22 = 0.82, recall = 18/22 = 0.82, F1 = 0.82.

Metrics for a registry

- **Duplicate rate:** extra entries per real object (lower is better).
- **Identity consistency:** fraction of an object's observations assigned to its main ID.
- **ID switches:** times an object's observations jump to another ID.

IoU and mIoU for maps

For each class, IoU = TP / (TP + FP + FN) over voxels (or pixels). **mIoU** is the mean over classes, so rare classes count as much as common ones. "Ignore" labels exclude regions without ground truth.

Fair evaluation

- Only count objects that were actually observable (in range, large enough, seen enough).
- Average over several random seeds when randomness is involved; report mean (and spread).
- Use the same inputs for every method so the comparison isolates the method.

In GeoSemTwin-X

Assets count as visible only if ≤ 8 m away, ≥ 0.2% of the image and in ≥ 2 keyframes. Synthetic results are the mean of 3 seeds.

Check yourself

25.1 A system reports 10 changes, 6 real, and misses 2: precision and recall? 25.2 Why is mIoU averaged over classes instead of over all voxels?

---

26. Experiments: baselines, ablations, sensitivity

How a research prototype proves its claims.

Baselines

A **baseline** is a simpler or established method run on exactly the same data. Results only mean something compared to baselines. Good baselines are fair (well-tuned) and representative of what people actually use.

Published comparisons

For standard benchmarks (like TUM RGB-D), you can quote numbers from other papers, clearly marked as published, as long as the sequence and metric are the same.

Ablation studies

Remove one component at a time and measure the drop. This shows **why** the method works and that every part is needed. If removing a part changes nothing, it is not contributing on that data, which is also worth reporting.

Sensitivity and robustness studies

Vary one condition (detector threshold, keyframe interval, injected pose noise) and plot how the metric responds. A method whose advantage grows with noise is robust.

Reporting honestly

- Report where your method ties or loses, and explain why.
- Separate the effects: e.g. evaluate fusion with ground-truth poses to isolate it from SLAM errors.
- State limitations with numbers and a plan.

In GeoSemTwin-X

Baselines: per-frame 2D labelling, centroid tracking, geometry-only clustering, nearest-frontier and random exploration, naive fusion, change detection without verification. Ablations: SLAM (no PnP, no loops) and GASF (no geometry, no multi-view gate, no merge, no confidence weighting). Sensitivity: threshold, keyframe interval, pose noise 0–40 cm.

Check yourself

26.1 What is the difference between a baseline and an ablation? 26.2 Why evaluate fusion with ground-truth poses too?

27. ROS 2: nodes, topics, TF, RViz, Nav2

The standard software framework that connects sensors, algorithms and robots.

Core concepts

Concept	Meaning
Node	An independent program with one job (camera driver, SLAM, planner).
Topic	A named channel; nodes **publish** messages to it and others **subscribe**. Many-to-many, asynchronous.
Message	Typed data, e.g. `sensor_msgs/Image`, `sensor_msgs/PointCloud2`, `geometry_msgs/PoseStamped`, `visualization_msgs/MarkerArray`, `nav_msgs/OccupancyGrid`.
Service	Request–reply call, e.g. "save the twin now".
Parameter	Configurable setting of a node, often loaded from a YAML file.
QoS	Delivery settings: reliable vs best effort, and "latched" (transient local) so late subscribers still get the last message.
TF (tf2)	Tracks how all coordinate frames relate over time (map → odom → base_link → camera_optical).
Time synchronisation	`message_filters` pairs messages with nearly equal timestamps (e.g. RGB with depth).
RViz2	3D viewer for topics: point clouds, markers, paths, grids, images.
Nav2	Navigation stack: accepts a goal pose and drives the robot there, avoiding obstacles.
Distribution	Named releases (Humble for Ubuntu 22.04, Jazzy for Ubuntu 24.04).



Publish–subscribe: nodes only agree on topic names and message types, so parts can be swapped (dataset player ↔ real camera).

Useful commands

`ros2 topic list`, `ros2 topic echo /geosemtwinx/status`, `ros2 topic hz /dataset/rgb`, `ros2 node list`, `ros2 service call /geosemtwinx/save_twin std_srvs/srv/Trigger`, `rviz2`.

In GeoSemTwin-X

A dataset player or a RealSense driver publishes RGB-D; the GeoSemTwin-X node publishes pose, path, map, assets, traversability, NBV goal, annotated image and status, plus TF map → camera_optical. Configs are YAML parameter files; `run_demo.sh` starts everything.

Check yourself

27.1 Topic or service for a continuous camera stream? 27.2 Why does the initial pose use a "latched" topic?

28. The tool chain

The software around the algorithms, and why each piece is there.

Tool	What it is	Why it matters here
Linux / Ubuntu	Operating system used by almost all robotics software	ROS 2, CUDA tools and research code target Ubuntu
WSL2	Real Linux kernel inside Windows, with GPU access	Run Ubuntu tools on a Windows laptop; several distros can coexist
Terminal / bash	Text commands (`cd` , `ls` , `cp` , `source`)	Every step is a command; PowerShell and bash have different syntax
Python	Main language for research code	All GeoSemTwin-X modules
venv / pip	Isolated Python environment and package installer	Keeps project packages separate from the system; activate with `source .venv/bin/activate`
conda	Another environment manager, also handles non-Python libraries	VIGS-SLAM's environment
CUDA / nvidia-smi	NVIDIA GPU platform / monitoring tool	GPU inference; power and memory measurements
PyTorch	Deep-learning library	Runs Grounding DINO and SAM 2.1
Hugging Face Hub	Repository of models and datasets; `hf auth login`	Downloads the models and the Staer dataset (gated: requires accepting terms)
Open3D	3D data library (point clouds, odometry, pose graphs)	SLAM building blocks
OpenCV	Classic computer-vision library	ORB features, PnP, image drawing
NumPy / SciPy	Numerical arrays / scientific algorithms	Voxel maps, Hungarian algorithm, rotations
ffmpeg	Video encoder/decoder	Decoding dataset videos; writing the demo video
Plotly / Matplotlib	Interactive / static plotting	Dashboard 3D views; result charts
Git / GitHub	Version control and code hosting	Getting VIGS-SLAM's official code
VS Code	Code editor with a WSL connection	Edit files and run terminals inside Ubuntu

Common beginner pitfalls (all seen during this project)

- Typing Linux commands in PowerShell (or the reverse): check the prompt (`PS …>` vs `user@host:~$`).
- Forgetting to activate the environment: "python: command not found" or "externally managed environment".
- Spaces in paths: wrap them in quotes.
- Ctrl+Z pauses a program; Ctrl+C stops it.
- Old system packages leaking into an environment (the brotli error): upgrade inside the environment.

Answers to "check yourself"

Q	Answer
1.1	It does not stay in sync with reality, so it cannot answer "what is there now".
1.2	Any two of: semantics (labels), stable identities, versions/history, confidence.
2.1	640 × 480 × 3 = 921,600.
2.2	Small changes in a depth value change the 3D position; colour can tolerate small errors visually.
3.1	u = 500·1/2 + 320 = 570.
3.2	u = 500·1/4 + 320 = 445: further away, closer to the centre.
4.1	Its columns are perpendicular unit vectors, so $R^T R = I$.
4.2	$p_{camera} = T_{W \leftarrow C}^{-1} \cdot p_{world}$.
5.1	$(6/2)^2 = 9$ times larger: about 9 cm.
5.2	A white wall has no texture to match between the two images; the projector adds texture.
6.1	X = (420 − 320)·2/500 = 0.4 m, Y = 0, Z = 2 m.
6.2	A few stray points (noise, mask leakage) would otherwise make the box far too big.
7.1	Position needs double integration, so small biases grow quadratically into large errors.
7.2	Gravity: about 9.81 m/s^2 pointing up in the sensor frame.
8.1	A corner is unique in both directions; a blank wall looks the same everywhere.
8.2	Two points.
9.1	Moving along the corridor does not change the geometry, so that direction is unconstrained.
9.2	That motion direction is well constrained (low uncertainty).
10.1	Errors are not added every frame, only when a new keyframe is created.
10.2	1 / 50 = 2%.
11.1	That two distant poses in the graph are actually close, which reveals and corrects accumulated drift.
11.2	A pose graph optimises poses only; bundle adjustment also optimises 3D points using pixel reprojection errors.
12.1	Scaling the scene and the motion together produces identical images.
12.2	Helps: speeding up, slowing down, turning. Does not: constant speed in a straight line.
13.1	The estimate and ground truth use different origins and orientations.
13.2	The estimated trajectory already has the correct metric size.
14.1	"Unknown" means not observed; you cannot claim an object is gone from a place you did not see.
14.2	Log-odds 0.
15.1	Equal weights: 2.05 m, with $\sigma = 2/\sqrt{2} \approx 1.4$ cm.
15.2	Observations can legitimately be further from the estimate when the pose is uncertain.
16.1	Training adjusts weights using labelled data; inference uses fixed weights to make predictions.

16.2	Each number uses 16 bits instead of 32.
17.1	Intersection 50, union 150: IoU = 1/3.
17.2	It reports the same object in every frame, with no link between frames.
18.1	Linking words or phrases in the text to regions in the image.
18.2	Closed-set: collect and label images, retrain. Grounding DINO: add "roll cage." to the prompt.
19.1	A box includes background pixels that would add wrong 3D points.
19.2	A prompt: a point, a box or a mask.
20.1	Mask edges often leak onto far background; the object is the near cluster.
20.2	Only the visible side of the object produces points.
21.1	It starts a new candidate object.
21.2	Drift moves the observed centre away from the stored centre, beyond the matching distance.
22.1	Recall: objects seen only once are never confirmed.
22.2	Confident detections are more often correct, so they should dominate the label.
23.1	The camera may simply not have looked there.
23.2	Usually a little recall, in exchange for much higher precision.
24.1	The boundary between known free space and unknown space.
24.2	An overestimate can make A* skip the true shortest path.
25.1	Precision 6/10 = 0.6; recall 6/(6+2) = 0.75.
25.2	So that rare classes (few voxels) count as much as common ones like the floor.
26.1	A baseline is a different method; an ablation is your method with one part removed.
26.2	To separate fusion errors from localisation errors.
27.1	A topic.
27.2	So a node that starts later still receives the last published value.

Further reading (free, beginner-friendly)

- R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. (free online): cameras, features, 3D.
- C. Stachniss, "SLAM" and "Photogrammetry" lecture videos (YouTube, University of Bonn).
- S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*: occupancy grids, uncertainty, exploration.
- The ROS 2 documentation tutorials (docs.ros.org): nodes, topics, TF, RViz.
- Original papers: Grounding DINO (ECCV 2024), SAM 2 (2024), ORB-SLAM2 (T-RO 2017), TUM RGB-D benchmark (IROS 2012).